



# Linux for Beginners

A Practical Guide

Chapter 7 of 12

**Processes & System Monitoring**

# Contents

---

|             |                                                         |
|-------------|---------------------------------------------------------|
| <b>7.1</b>  | What Is a Process?                                      |
| <b>7.2</b>  | Viewing Running Processes with ps                       |
| <b>7.3</b>  | Interactive Monitoring with top                         |
| <b>7.4</b>  | htop: A Better top                                      |
| <b>7.5</b>  | Sending Signals to Processes: kill, killall, pkill      |
| <b>7.6</b>  | Foreground, Background, and Job Control                 |
| <b>7.7</b>  | Running Processes That Survive Logout: nohup and screen |
| <b>7.8</b>  | Monitoring System Resources                             |
| <b>7.9</b>  | Disk Usage: df and du                                   |
| <b>7.10</b> | Understanding Load Average                              |
| <b>7.11</b> | Practical Monitoring Scenarios                          |
| <b>7.12</b> | Essential Commands: Quick Reference                     |
| <b>7.13</b> | Chapter Summary                                         |

---

## Chapter 7

# Processes & System Monitoring

## 7.1 What Is a Process?

Every time you run a command, open an application, or start a service, Linux creates a **process** — a running instance of a program, with its own chunk of memory, its own identity, and its own lifecycle. Understanding processes is fundamental to understanding what your system is actually doing at any given moment.

Every process in Linux has several key attributes:

- **PID** (Process ID) — a unique number that identifies the process. Every process gets one.
- **PPID** (Parent Process ID) — the PID of the process that spawned this one. All processes descend from a single ancestor.
- **Owner** — the user account the process runs under. A process inherits the permissions of its owner.
- **State** — what the process is currently doing: running, sleeping, stopped, or zombie.
- **Priority / nice value** — how much CPU time the process gets relative to others.

### The process tree

Linux processes form a tree. The root of the tree is `systemd` (PID 1) — the first process started at boot. Every other process is a descendant of PID 1. You can view the whole tree with:

```
$ pstree
systemd┌───ModemManager─2*[{ModemManager}]
      │   └──NetworkManager─2*[{NetworkManager}]
      │   └──bash─pstree
      │   └──cron
      └──sshd─sshd─bash
```

bash

### Process states

| State      | Symbol | Meaning                                         |
|------------|--------|-------------------------------------------------|
| Running    | R      | Actively using CPU right now                    |
| Sleeping   | S      | Waiting for something (I/O, timer, user input)  |
| Disk sleep | D      | Uninterruptible sleep — waiting on hardware I/O |
| Stopped    | T      | Paused (e.g. with Ctrl+Z)                       |

| State  | Symbol | Meaning                                              |
|--------|--------|------------------------------------------------------|
| Zombie | Z      | Finished but parent hasn't collected exit status yet |

## 7.2 Viewing Running Processes with ps

**ps** (**p**rocess **s**tatus) takes a snapshot of running processes at the moment you run it. Unlike **top**, it doesn't update — it prints and exits. This makes it ideal for scripting and filtering.

```
# Show your own processes in this terminal:
```

```
$ ps
```

```
  PID TTY          TIME CMD
 1234 pts/0    00:00:00 bash
 1891 pts/0    00:00:00 ps
```

```
# Show ALL processes on the system (most common usage):
```

```
$ ps aux
```

```
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1   0.0  0.1 168784 11824 ?        Ss   09:00   0:01 /sbin/init
root       423   0.0  0.0  14864  1024 ?        Ss   09:00   0:00 /sbin/sshd
ashik    1234   0.0  0.0  22132  5120 pts/0    Ss   09:05   0:00 -bash
ashik    1891   0.0  0.0  14344  3584 pts/0    R+   09:10   0:00 ps aux
```

### Reading ps aux output

| Column  | Meaning                                                       |
|---------|---------------------------------------------------------------|
| USER    | The user who owns the process                                 |
| PID     | Process ID — unique identifier                                |
| %CPU    | Percentage of CPU currently used                              |
| %MEM    | Percentage of RAM currently used                              |
| VSZ     | Virtual memory size (KB) — total memory the process could use |
| RSS     | Resident Set Size — actual RAM currently in use (KB)          |
| STAT    | Process state (R=running, S=sleeping, Z=zombie, T=stopped)    |
| START   | When the process was started                                  |
| TIME    | Total CPU time consumed                                       |
| COMMAND | The command that launched the process                         |

## Filtering ps output with grep

```
# Find a specific process by name:
$ ps aux | grep nginx
root      1423  0.0  0.1 55348 4096 ?  Ss  09:00   0:00 nginx: master
www-data  1424  0.0  0.0 55672 2048 ?   S   09:00   0:00 nginx: worker

# Find processes owned by a specific user:
$ ps aux | grep ashik

# Show process tree with ps:
$ ps axjf
```

bash

## 7.3 Interactive Monitoring with top

top shows a live, continuously updating view of running processes sorted by CPU usage. It's available on every Linux system without installation.

```
$ top

top - 10:42:01 up 1:42,  1 user,  load average: 0.15, 0.12, 0.09
Tasks: 142 total,  1 running, 141 sleeping,  0 stopped,  0 zombie
%Cpu(s):  2.3 us,  0.7 sy,  0.0 ni, 96.8 id,  0.2 wa,  0.0 hi,  0.0 si
MiB Mem :  7865.3 total,  4201.2 free,  2104.8 used,  1559.3 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used.  5448.2 avail Mem

   PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
  1423 root        20   0 612344  42112 28672 S   8.6   0.5   0:12.34  nginx
  1156 ashik       20   0 3241984 312320 98304 S   4.3   3.9   2:45.67  firefox
  1234 ashik       20   0   22132   5120  4096 S   0.0   0.1   0:00.04   bash
```

bash

### top keyboard shortcuts

| Key | Action                                       |
|-----|----------------------------------------------|
| q   | Quit top                                     |
| k   | Kill a process — prompts for PID then signal |
| r   | Renice a process — change its priority       |
| M   | Sort by memory usage                         |
| P   | Sort by CPU usage (default)                  |
| T   | Sort by total running time                   |

| Key   | Action                                     |
|-------|--------------------------------------------|
| u     | Filter by username                         |
| 1     | Toggle per-CPU breakdown (shows each core) |
| h     | Help screen                                |
| Space | Refresh immediately                        |

## 7.4 htop: A Better top

htop is an improved, colour-coded, mouse-enabled version of top. It's not installed by default on all systems but is always worth installing. It shows CPU cores as individual bars, memory as a visual gauge, and lets you scroll through processes and kill them with a single keypress.

```
$ sudo apt install htop
$ htop
```

bash

| Key      | Action                                |
|----------|---------------------------------------|
| F2       | Setup / configure htop display        |
| F3 or /  | Search for a process by name          |
| F4       | Filter processes                      |
| F5       | Toggle tree view                      |
| F6       | Sort by column                        |
| F9       | Kill selected process (choose signal) |
| F10 or q | Quit                                  |
| Space    | Tag a process (for bulk actions)      |
| u        | Filter by user                        |

**TIP**

Install htop on every Linux system you use. It's the single most useful monitoring tool for day-to-day work — one glance tells you everything about system health.

## 7.5 Sending Signals to Processes: kill, killall, pkill

In Linux, you control processes by sending them **signals** — predefined messages that tell a process to do something. Despite the name, `kill` doesn't only terminate processes — it sends any signal you choose.

### Common signals

| Signal  | Number | Default Action          | When to Use                                         |
|---------|--------|-------------------------|-----------------------------------------------------|
| SIGTERM | 15     | Graceful termination    | First choice — lets process clean up before exiting |
| SIGKILL | 9      | Immediate termination   | Last resort — cannot be caught or ignored           |
| SIGHUP  | 1      | Reload configuration    | Tell a daemon to re-read its config file            |
| SIGSTOP | 19     | Pause the process       | Same as Ctrl+Z                                      |
| SIGCONT | 18     | Resume a paused process | Resume after SIGSTOP                                |
| SIGINT  | 2      | Interrupt               | Same as Ctrl+C                                      |

### kill — send a signal by PID

```
# First, find the PID:
$ ps aux | grep firefox
ashik      1156  4.3  3.9  ...  firefox

# Send SIGTERM (graceful) – default signal:
$ kill 1156

# If it doesn't respond, send SIGKILL (force):
$ kill -9 1156

# Send SIGHUP to reload a daemon's config:
$ sudo kill -1 $(pgrep nginx)
```

bash

### killall and pkill — send signals by name

```
bash
# killall matches exact process name:
$ killall firefox      # send SIGTERM to all firefox processes
$ killall -9 firefox    # force kill all firefox processes

# pkill is more flexible – matches partial names and supports flags:
$ pkill firefox        # same as killall firefox
$ pkill -u ashik        # kill all processes owned by user ashik
$ pkill -f 'python script' # match against full command line
```

**TIP**

Always try SIGTERM (kill PID or kill -15 PID) first and wait a few seconds. Only use SIGKILL (-9) if the process refuses to respond — it gives the process no chance to save data or clean up temporary files.

## 7.6 Foreground, Background, and Job Control

By default, when you run a command, it runs in the **foreground** — it occupies your terminal until it finishes. Linux lets you move processes between foreground and background, so you can keep working while long-running tasks continue.

### Running a process in the background

```
bash
# Append & to run a command in the background immediately:
$ sleep 60 &
[1] 2341
# [1] is the job number, 2341 is the PID

# List all background jobs in this shell session:
$ jobs
[1]+  Running    sleep 60 &

# Bring job 1 to the foreground:
$ fg %1

# Pause a foreground process and send it to background:
# (while it's running, press:)
Ctrl + Z
# Output: [1]+  Stopped    sleep 60

# Resume it in the background:
$ bg %1
[1]+ sleep 60 &
```

### Job control summary



| Action                   | Command / Key          |
|--------------------------|------------------------|
| Run in background        | command &              |
| Pause foreground process | Ctrl + Z               |
| Resume in background     | bg %N (N = job number) |
| Bring to foreground      | fg %N                  |
| List all jobs            | jobs                   |
| Kill a background job    | kill %N                |

## 7.7 Running Processes That Survive Logout: nohup and screen

Background jobs launched with & are still tied to your terminal session. When you log out, the shell sends **SIGHUP** to all its child processes — and they die. For long-running tasks on remote servers, you need a way to keep processes alive after you disconnect.

### nohup — immune to hangup

```
# Run a script that keeps going after you log out:
$ nohup python3 long_task.py &
nohup: ignoring input and appending output to 'nohup.out'
[1] 3421

# Output goes to nohup.out by default – redirect if needed:
$ nohup python3 long_task.py > output.log 2>&1 &
```

bash

### screen — persistent terminal sessions

screen creates a persistent terminal session you can detach from and reattach to — even after logging out and back in:

```
bash
$ sudo apt install screen

# Start a new named screen session:
$ screen -S mysession

# Run your long task inside the screen session:
$ python3 long_task.py

# Detach from screen (process keeps running):
Ctrl + A, then D

# List running screen sessions:
$ screen -ls
There is a screen on:
  3421.mysession  (Detached)

# Reattach to the session:
$ screen -r mysession
```

**TIP**

For modern systems, tmux is a more powerful alternative to screen with better split-pane support. Install with: `sudo apt install tmux`

## 7.8 Monitoring System Resources

Beyond processes, you need to understand how your system's resources — CPU, memory, and I/O — are being consumed. Linux provides several purpose-built tools for this.

### free — memory usage

```
bash
$ free -h
```

|       | total | used  | free  | shared | buff/cache | available |
|-------|-------|-------|-------|--------|------------|-----------|
| Mem:  | 7.7Gi | 2.1Gi | 4.1Gi | 312Mi  | 1.5Gi      | 5.0Gi     |
| Swap: | 2.0Gi | 0B    | 2.0Gi |        |            |           |

The **available** column (not **free**) is what matters — it shows how much memory is actually usable for new processes. Linux aggressively uses free RAM as disk cache (**buff/cache**), which it releases when applications need it.

### vmstat — virtual memory statistics

```

# Show a snapshot:
$ vmstat
procs -----memory----- --swap-- -----io----- -system-- -----cpu-----
 r  b   swpd   free   buff   cache   si   so    bi    bo    in   cs us sy id wa
  1   0       0 4194304 102400 1536000    0    0    12     8   240  480  2  1 97  0

# Update every 2 seconds:
$ vmstat 2

```

## iostat — CPU and disk I/O statistics

```

$ sudo apt install sysstat

# Show CPU and disk stats:
$ iostat
Linux 5.15.0 (ubuntu) 06/10/2024 _x86_64_ (4 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           2.30    0.00    0.70    0.20    0.00   96.80

Device            tps    kB_read/s    kB_wrtn/s
sda                4.20        48.00        32.00

# Live update every 2 seconds:
$ iostat 2

```

## 7.9 Disk Usage: df and du

Two tools cover disk usage from different angles: `df` shows how full each filesystem is; `du` shows how much space a specific directory or file is using.

### df — disk filesystem usage

```

$ df -h

```

| Filesystem | Size | Used | Avail | Use% | Mounted on |
|------------|------|------|-------|------|------------|
| /dev/sda1  | 20G  | 12G  | 7.2G  | 63%  | /          |
| tmpfs      | 3.9G | 0    | 3.9G  | 0%   | /dev/shm   |
| /dev/sda2  | 100G | 45G  | 50G   | 47%  | /home      |

```

# Show filesystem type too:
$ df -hT

```

### du — directory space usage

```

# Show size of a directory and all its contents:
$ du -sh ~/Documents
4.2G    /home/ashik/Documents

# Show sizes of each subdirectory:
$ du -h --max-depth=1 ~
512M    /home/ashik/Downloads
4.2G    /home/ashik/Documents
128M    /home/ashik/.config
4.9G    /home/ashik

# Find the top 10 largest items in a directory:
$ du -ah /var/log | sort -rh | head -10

```

## 7.10 Understanding Load Average

The **load average** is one of the most important health indicators on a Linux system. It appears in the output of `top`, `htop`, and `uptime`, and tells you how busy the system has been.

```

$ uptime
10:42:01 up 1:42,  1 user,  load average: 0.15, 0.12, 0.09
#                                     ^1min ^5min ^15min

```

The three numbers represent the average number of processes that were either running or waiting to run over the last **1 minute**, **5 minutes**, and **15 minutes**.

### Interpreting load average

| Load Average | On a 1-core system | On a 4-core system | Interpretation                          |
|--------------|--------------------|--------------------|-----------------------------------------|
| < 1.0        | Good               | Very light         | System is idle — plenty of capacity     |
| 1.0          | 100% busy          | 25% busy           | At capacity for 1 core                  |
| 2.0 – 4.0    | Overloaded         | 50–100%            | Watch closely on 4-core, fine on 8-core |
| > cores      | Overloaded         | Overloaded         | Processes are queueing — investigate    |

#### TIP

A load average equal to your CPU core count means the system is fully utilised — not necessarily a problem. A load average significantly higher than your core count for a sustained period means something needs attention.

## 7.11 Practical Monitoring Scenarios

### Scenario 1: Find what's eating your CPU

```
# Open htop sorted by CPU:
$ htop
# Press P to sort by CPU – the top process is your culprit

# Or with ps:
$ ps aux --sort=-%cpu | head -10
```

bash

### Scenario 2: Find what's eating your memory

```
$ ps aux --sort=-%mem | head -10

# Also useful – see memory totals at a glance:
$ free -h
```

bash

### Scenario 3: A process won't die

```
# Find the PID:
$ pgrep -l stuck_process
3421 stuck_process

# Try graceful termination first:
$ kill 3421

# Wait 5 seconds. If still running, force kill:
$ kill -9 3421

# If you don't know the PID:
$ pkill -9 stuck_process
```

bash

### Scenario 4: Disk is full — find what's using space

```
bash
# See which filesystem is full:
$ df -h

# Find the biggest directories:
$ du -ah / --max-depth=3 2>/dev/null | sort -rh | head -20

# Common culprits to check:
$ du -sh /var/log/          # logs
$ du -sh /tmp/              # temp files
$ sudo du -sh /var/cache/apt/ # apt cache (safe to clear)
$ sudo apt clean            # clear apt cache
```

### Scenario 5: System feels slow — quick diagnosis

```
bash
# Check load average and uptime:
$ uptime

# Check if it's CPU, memory, or I/O:
$ vmstat 1 5    # watch for high 'wa' (I/O wait) or 'us' (user CPU)

# Check if swap is being used heavily (sign of memory pressure):
$ free -h      # if swap used > 0, you may be low on RAM
```

## 7.12 Essential Commands: Quick Reference

| Command      | What It Does                       | Common Usage                 |
|--------------|------------------------------------|------------------------------|
| ps aux       | Snapshot of all running processes  | ps aux   grep nginx          |
| ps axjf      | Process snapshot in tree format    | ps axjf                      |
| pstree       | Visual process tree                | pstree -p                    |
| pgrep name   | Find PID(s) of processes by name   | pgrep nginx                  |
| top          | Live process monitor (built-in)    | top                          |
| htop         | Improved live process monitor      | htop                         |
| kill PID     | Send SIGTERM to a process          | kill 1234                    |
| kill -9 PID  | Force kill a process (SIGKILL)     | kill -9 1234                 |
| kill -1 PID  | Send SIGHUP (reload config)        | sudo kill -1 \$(pgrep nginx) |
| killall name | Kill all processes with exact name | killall firefox              |

| Command                      | What It Does                             | Common Usage                         |
|------------------------------|------------------------------------------|--------------------------------------|
| <code>pkill name</code>      | Kill processes matching name pattern     | <code>pkill -9 stuck_app</code>      |
| <code>command &amp;</code>   | Run command in background                | <code>python3 server.py &amp;</code> |
| <code>Ctrl + Z</code>        | Pause foreground process                 | (keyboard shortcut)                  |
| <code>bg %N</code>           | Resume job N in background               | <code>bg %1</code>                   |
| <code>fg %N</code>           | Bring job N to foreground                | <code>fg %1</code>                   |
| <code>jobs</code>            | List background jobs                     | <code>jobs</code>                    |
| <code>nohup cmd &amp;</code> | Run command immune to logout             | <code>nohup ./task.sh &amp;</code>   |
| <code>screen -S name</code>  | Create persistent terminal session       | <code>screen -S server</code>        |
| <code>screen -r name</code>  | Reattach to a screen session             | <code>screen -r server</code>        |
| <code>free -h</code>         | Show memory usage in human-readable form | <code>free -h</code>                 |
| <code>vmstat</code>          | Virtual memory and CPU statistics        | <code>vmstat 2</code>                |
| <code>iostat</code>          | CPU and disk I/O statistics              | <code>iostat 2</code>                |
| <code>df -h</code>           | Show disk filesystem usage               | <code>df -h</code>                   |
| <code>du -sh dir</code>      | Show size of a directory                 | <code>du -sh ~/Documents</code>      |
| <code>uptime</code>          | Show uptime and load average             | <code>uptime</code>                  |

## 7.13 Chapter Summary

- ✓ Every running program is a **process** with a unique PID, a parent (PPID), an owner, and a state. All processes descend from PID 1 (`systemd`).
- ✓ `ps aux` takes a snapshot of all processes. Pipe it through `grep` to find specific ones. `ps axjf` shows the tree.
- ✓ `top` is always available for live monitoring. `htop` is better — install it on every system.
- ✓ Signals control processes. Always try **SIGTERM** (kill PID) first. Only use **SIGKILL** (-9) as a last resort — it gives the process no chance to clean up.
- ✓ `kill` sends a signal by PID. `killall` targets by exact name. `pkill` is the most flexible — it matches patterns.

- ✓ Use `&` to run in background, `Ctrl+Z` to pause, `bg` to resume in background, `fg` to bring to front.
- ✓ `nohup` keeps a process alive after logout. `screen` (or `tmux`) creates persistent sessions you can detach and reattach.
- ✓ `free -h` shows memory; the **available** column is what matters. `vmstat` and `iostat` show CPU and I/O pressure.
- ✓ `df -h` shows how full filesystems are. `du -sh dir` shows how much space a directory uses. Combine `du | sort -rh | head` to find space hogs.
- ✓ Load average compares against CPU core count: `load < cores` = healthy; `load > cores` sustained = investigate.

---

## What's Coming in Chapter 8

With system monitoring mastered, Chapter 8 moves to the network — how your Linux machine talks to the world and how you control it. You'll learn:

- How Linux networking works: interfaces, IP addresses, and routing
- Essential network tools: `ip`, `ping`, `curl`, `wget`, `netstat`, `ss`
- Connecting to remote servers with SSH — keys, config, and secure tunnels
- Transferring files over the network with `scp` and `rsync`
- Checking DNS, diagnosing network problems, and reading network interfaces
- Firewalls with `ufw` — the beginner-friendly firewall for Ubuntu

*Chapter 8 is where Linux becomes a tool for the internet.*



---

Author

**Ashik A**

[github.com/ashihh07](https://github.com/ashihh07)

---

This guide was created, organized, refined, and designed by Ashik A using GitHub documentation, industry best practices, publicly available learning resources, and AI-assisted research.

---